

Objectif : Découvrir les requêtes asynchrones en Ajax, mettre en œuvre les bonnes pratiques de développement javascript (fonctions anonymes, arguments passés comme des objets JSON...)

Exercice 1 : Rappels de javascript = librairie utils.js

- Réaliser une fonction trace
 - Utilise console.log si le navigateur n'est pas Internet Explorer
- Réaliser une fonction debug(s)
 - Cette fonction ne fait plus rien après un nombre d'appels donné
 - Elle affiche la valeur du compteur si on ne lui fournit pas d'argument
- **Réaliser la même fonction en utilisant une fermeture (closure) permettant de protéger l'accès aux variables manipulées**
- Réaliser les fonctions show, hide, html, val, check
 - Leurs arguments peuvent être des identifiants ou des références : utiliser typeof
- Transformer votre code en une librairie de fonctions

Exercice 2 : Fonctions de rappel, JSON = librairie boucle.js

- Déclencher un traitement périodique boucle(iPeriode,fCbTraitement,fCbContinuer) :
 - qui effectue un appel à une fonction callback de traitement
 - puis un appel à une fonction gérant la poursuite de boucle
 - On n'utilisera pas les arguments supplémentaires de window.setTimeout
- Transformer votre code en une librairie de fonctions
 - Comment rendre le 3ème argument optionnel ?
 - Passer les arguments en JSON
 - **Comment sont manipulés les types objet dans le cadre d'affectations en js ?**
 - **Comment dupliquer/cloner un objet en js ?**
 - **Utiliser pour cela une fonction « enrichir(oDef,oModif) »**
 - [TNE] Proposer une fonction boucle admettant les deux modes de passage de paramètres
 - **Protéger les variables globales à l'aide d'une fermeture**

Exercice 3 : Cas d'étude TryIT, fonctions anonymes

- Refaire TryIT Editor (L1) à l'aide de boucle:
 - Un champ textarea permet d'éditer du code HTML
 - Un div placé à côté montre le rendu suite à un clic
 - Ou à intervalles réguliers (fonctionnalité débrayable)
 - Un appui sur la touche Esc annule la mise à jour automatique
- Utiliser html, val, show, check, hide, boucle
 - Utiliser une fonction anonyme pour la fonction de poursuite

Exercice 4 : Objet XMLHttpRequest : méthode onreadystatechange

- A1_traitReponse.html :
 - Tester la récupération d'un mot de façon asynchrone
 - Décommentez l'appel à alert dans la fonction traiteReponse
 - Que remarquez vous ?

Exercice 5 : Objet XMLHttpRequest : envoi de valeurs au serveur

- A2_EnvoiRequete.html :
 - Permettre à l'utilisateur de saisir une : clé dans un champ d'entrée texte
 - Créer un bouton pour déclencher la requête asynchrone envoyant la clé saisie par l'utilisateur au serveur
 - Placer les données récupérées dans un div côté client
- A2_data.php
 - Définir un tableau associatif dans data.php
 - Le serveur renvoie la valeur associée à la clé transmise

Exercice 6 : Objet XMLHttpRequest : refactoring de code = librairie ajax.js

- Améliorer les fonctions envoiRequete et traiteReponse pour que la fonction de callback choisie soit destinataire de la réponse du serveur comme paramètre, lorsque celle-ci est disponible
- L'intégrer dans une librairie de fonctions
 - Quelle restriction entraîne votre solution technique ?
- Définir un handler 'ajax' pour la fonction envoiRequete
 - Il admettra un paramètre sous forme de JSON
 - type, callback, data seront facultatifs, url devra être fourni
 - Les données seront passées par json également, par exemple {"debutNom":"T"} au lieu de «debutNom=T». Vous utiliserez la structure for ... in

Exercice 7 : Cas d'étude Web2 Suggest

- Un formulaire permet de saisir le début du nom de famille d'un étudiant dans un champ input
 - Les données sont stockées dans un fichier
 - Au fur et à mesure de la saisie, une liste de propositions s'affiche
 - On peut cliquer sur chaque proposition pour remplir le champ automatiquement
- IMPORTANT : l'expérience utilisateur !
 - Afficher un gif animé lors du déclenchement d'une requête asynchrone
 - Différer la réponse du serveur en utilisant sleep

Exercice 8 : Améliorations du Web2 Suggest

- Améliorer le Web2Suggest pour consommer un flux de données encodé en JSON avec plusieurs types de propriétés : tableau d'étudiants, avec leur genre, lus depuis un fichier
- Les données sont lues depuis une base de données avec les librairies de L1 (SQLSelect et ParcoursRs)
 - Utiliser json_encode pour transformer un tableau associatif en objet JSON
 - Comment ajouter des champs supplémentaires à un résultat MySQL pour préparer l'objet JSON ?
- Permettre de choisir le critère de recherche avec un paramètre action=parNom ou parPrenom
 - L'utilisateur pourra cliquer sur un bouton de configuration lui permettant de sélectionner ce sur quoi il effectue sa recherche
- On ne remplit pas forcément le champ avec ce qui est suggéré : notion de cache
 - Au fur et à mesure, le cache se remplit... éviter de faire des requêtes inutiles !
- Créer une fonction mkSelect qui remplit/fabrique le menu déroulant
 - Un premier argument sera un objet JSON contenant des données à afficher
 - Un objet JSON servira de configuration pour la fonction